

Практическая работа 32. Чтение и обновление данных с помощью Room

В предыдущих уроках вы узнали, как использовать библиотеку постоянства Room, слой абстракции поверх базы данных SQLite, для хранения данных приложения. В этом уроке вы добавите больше возможностей в приложение Inventory и узнаете, как читать, отображать, обновлять и удалять данные из базы данных SQLite с помощью Room. Вы будете использовать LazyColumn для отображения данных из базы данных и автоматического обновления данных при изменении базовых данных в базе данных.

Необходимые условия

- Умение создавать и взаимодействовать с базой данных SQLite с помощью библиотеки Room.
- Умение создавать сущность, DAO и классы базы данных.
- Уметь использовать объект доступа к данным (DAO) для отображения функций Kotlin в SQL-запросы.
- Умение отображать элементы списка в LazyColumn. Завершение предыдущего урока в этом блоке, Persist data with Room.

Что вы узнаете

- Как читать и отображать сущности из базы данных SQLite.
- Как обновлять и удалять сущности из базы данных SQLite с помощью библиотеки Room.

Что вы создадите

- Приложение Inventory, которое отображает список инвентарных объектов и может обновлять, редактировать и удалять объекты из базы данных приложения с помощью библиотеки Room.

2. Starter app overview

This codelab uses the Inventory app solution code from the previous codelab, Persist data with Room as the starter code. The starter app already saves data with the Room persistence library. The user can use the Add Item screen to add data to the app database.

Note: The current version of the starter app doesn't display the data stored in the database.



In this codelab, you extend the app to read and display the data, and update and delete entities on the database using a Room library.

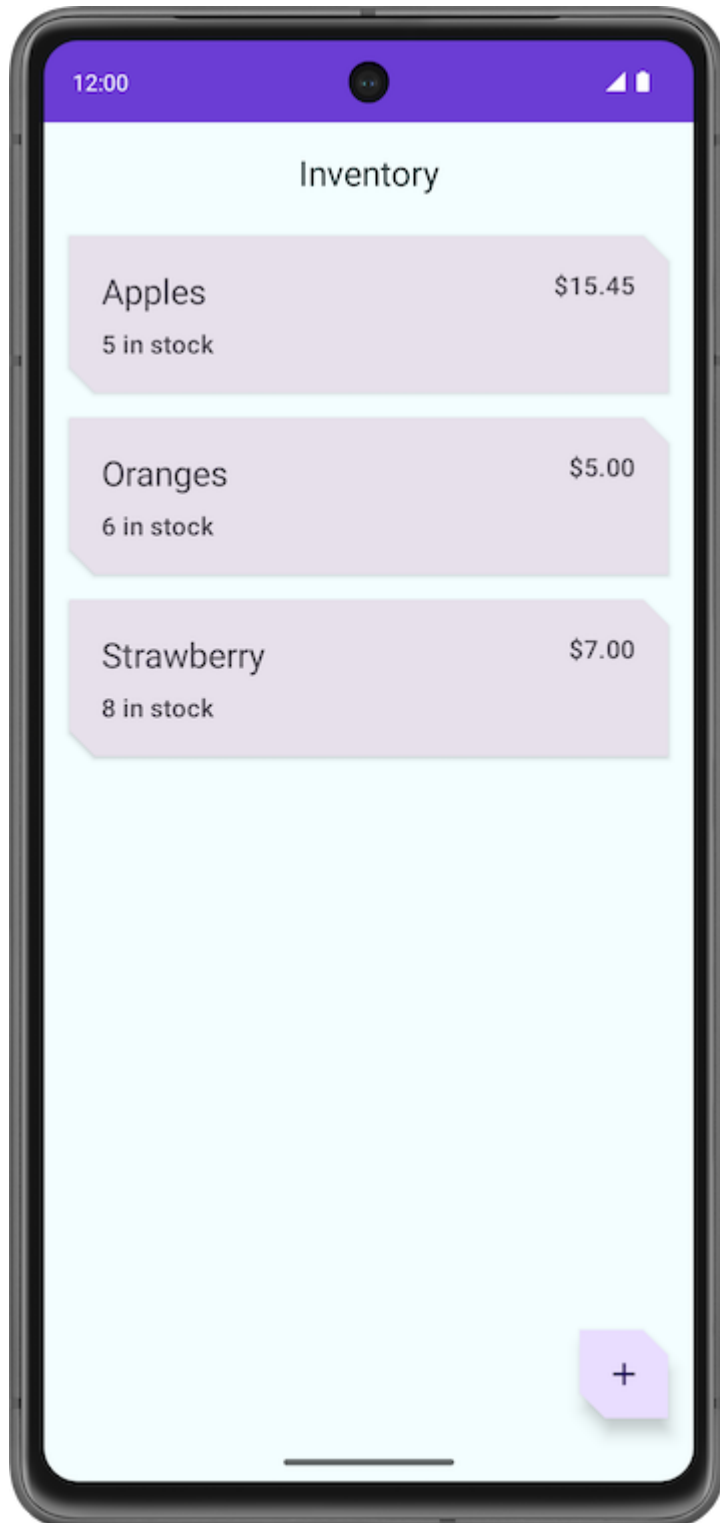
Download the starter code for this codelab To get started, download the starter code:

```
$ git clone https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app.git
```

```
$ cd basic-android-kotlin-compose-training-inventory-app  
$ git checkout room
```

3. Обновление состояния пользовательского интерфейса

В этой задаче вы добавляете в приложение LazyColumn для отображения данных, хранящихся в базе данных.



Прохождение композитной функции HomeScreen

Откройте файл `ui/home/HomeScreen.kt` и посмотрите на композитную функцию `HomeScreen()`.

```
kt @Composable fun HomeScreen( navigateToItemEntry: () -> Unit, navigateToItemUpdate: (Int) -> Unit,
    модификатор: Modifier = Modifier, ) { val scrollBehavior = TopAppBarDefaults.enterAlwaysScrollBehavior()
```

```
Scaffold(
    topBar = {
        // Верхнее приложение с заголовком приложения
    },
    floatingActionButton = {
        FloatingActionButton(
            // детали onClick
        ) {
            Icon(
                // детали иконки
            )
        }
    },
) { innerPadding ->

    // Отображение заголовка списка и списка элементов
    HomeBody(
        itemList = listOf(), // Для itemList передается пустой список
        onItemClick = navigateToItemUpdate,
        modifier = modifier.padding(innerPadding)
            .fillMaxSize()
    )
}
```

Эта композитная функция отображает следующие элементы:

Верхняя панель приложения с заголовком приложения

Плавающая кнопка действия (FAB) для добавления новых предметов в инвентарь

`7b1535d90ee957fa.png`

Составная функция `HomeBody()`

Функция `HomeBody()` `composable` отображает элементы инвентаря на основе переданного списка. Как часть реализации стартового кода, композитной функции

`HomeBody()` передается пустой список (`listOf()`). Чтобы передать список инвентаря в эту композитную функцию, необходимо получить данные об инвентаре из хранилища и передать их в модель `HomeViewModel`.

Передача состояния пользовательского интерфейса в `HomeViewModel`

Когда вы добавили в `ItemDao` методы для получения элементов - `getItem()` и `getAllItems()` - вы указали `Flow` в качестве возвращаемого типа. Напомним, что поток представляет собой общий поток данных. Возвращая поток, вам нужно только один раз

явно вызвать методы из DAO для данного жизненного цикла. Room обрабатывает обновления базовых данных асинхронным способом.

Получение данных из потока называется сбором из потока. При сборе данных из потока в слое пользовательского интерфейса необходимо учитывать несколько моментов.

События жизненного цикла, такие как изменение конфигурации, например поворот устройства, приводят к воссозданию активности. Это приводит к переконпозиции и сбору данных из потока заново.

Вы хотите, чтобы значения кэшировались как состояние, чтобы существующие данные не терялись между событиями жизненного цикла.

Потоки должны быть отменены, если не осталось наблюдателей, например, после завершения жизненного цикла композита.

Рекомендуемый способ отображения потока из ViewModel - это StateFlow.

Использование StateFlow позволяет сохранять и наблюдать данные независимо от жизненного цикла пользовательского интерфейса. Чтобы преобразовать поток в StateFlow, используется оператор stateIn.

Оператор stateIn имеет три параметра, которые описаны ниже:

scope - ViewModelScope определяет жизненный цикл StateFlow. Когда viewModelScope отменяется, StateFlow также отменяется.

started - Конвейер должен быть активен только тогда, когда виден пользовательский интерфейс. Для этого используется SharingStarted.WhileSubscribed(). Чтобы настроить задержку (в миллисекундах) между исчезновением последнего подписчика и остановкой корутины совместного доступа, передайте методу SharingStarted.WhileSubscribed() значение TIMEOUT_MILLIS.

initialValue - Установите начальное значение потока состояний в HomeUiState(). После преобразования потока в StateFlow вы можете собрать его с помощью метода collectAsState(), преобразовав его данные в State того же типа.

В этом шаге вы получите все элементы из базы данных Room в виде наблюдаемого API StateFlow для состояния пользовательского интерфейса. Когда данные Room Inventory изменятся, пользовательский интерфейс обновится автоматически.

Откройте файл ui/home/HomeViewModel.kt, который содержит константу TIMEOUT_MILLIS и класс данных HomeUiState со списком элементов в качестве параметра конструктора.

```
``kt
// Не нужно копировать, этот код является частью стартового кода

class HomeViewModel : ViewModel() {

    объект-компаньон {
        private const val TIMEOUT_MILLIS = 5_000L
    }
}

data class HomeUiState(val itemList: List<Item> = listOf())
```

Внутри класса `HomeViewModel` объявите `val` под названием `homeUiState` типа `StateFlow`. Вскоре вы решите проблему с ошибкой инициализации.

```
``kt val homeUiState: StateFlow
```

Вызовите `getAllItemsStream()` на `itemsRepository` и присвойте его `homeUiState`, который вы только что объявили.

```
``kt
val homeUiState: StateFlow<HomeUiState> =
    itemsRepository.getAllItemsStream()
```

Теперь вы получаете ошибку - `Unresolved reference: itemsRepository`. Чтобы устранить ошибку `Unresolved reference`, вам нужно передать объект `ItemsRepository` в `HomeViewModel`.

Добавьте параметр конструктора типа `ItemsRepository` в класс `HomeViewModel`.

```
``kt import com.example.inventory.data.ItemsRepository
```

```
class HomeViewModel(itemsRepository: ItemsRepository): ViewModel() {
```

В файле `ui/AppViewModelProvider.kt`, в инициализаторе `HomeViewModel`, передайте объект `ItemsRepository`, как показано на рисунке.

```
``kt
инициализатор {
    HomeViewModel(inventoryApplication().container.itemsRepository)
}
```

Вернитесь к файлу `HomeViewModel.kt`. Обратите внимание на ошибку несоответствия типов. Чтобы решить эту проблему, добавьте карту трансформации, как показано ниже.

```
``kt val homeUiState: StateFlow = itemsRepository.getAllItemsStream().map { HomeUiState(it) }
```

Android Studio по-прежнему показывает ошибку несоответствия типов. Эта ошибка связана с тем, что `homeUiState` имеет тип `StateFlow`, а `getAllItemsStream()` возвращает `Flow`.

Используйте оператор `stateIn`, чтобы преобразовать поток в `StateFlow`. `StateFlow` - это наблюдаемый API для состояния пользовательского интерфейса, который позволяет пользовательскому интерфейсу обновлять себя.

```
``kt
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.SharingStarted
```

```

import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.stateIn

val homeUiState: StateFlow<HomeUiState> =
    itemsRepository.getAllItemsStream().map { HomeUiState(it) }
        .stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(TIMEOUT_MILLIS),
            initialValue = HomeUiState()
        )

```

Соберите приложение, чтобы убедиться в отсутствии ошибок в коде. Визуальных изменений не будет.

4. Отображение данных инвентаря

В этой задаче вы собираете и обновляете состояние пользовательского интерфейса в HomeScreen.

В файле HomeScreen.kt, в композитной функции HomeScreen, добавьте новый параметр функции типа HomeViewModel и инициализируйте его.

```

import androidx.lifecycle.viewmodel.compose.viewModel
import com.example.inventory.ui.AppViewModelProvider

@Composable
fun HomeScreen(
    navigateToItemEntry: () -> Unit,
    navigateToItemUpdate: (Int) -> Unit,
    modifier: Modifier = Modifier,
    viewModel: HomeViewModel = viewModel(factory = AppViewModelProvider.Factory)
)

```

В составной функции HomeScreen добавьте val под названием homeUiState для сбора состояния пользовательского интерфейса из модели HomeViewModel. Вы используете функцию collectAsState(), которая собирает значения из этого StateFlow и представляет его последнее значение через State.

```
``kt import androidx.compose.runtime.collectAsState import androidx.compose.runtime.getValue
```

```
val homeUiState by viewModel.homeUiState.collectAsState()
```

Обновите вызов функции HomeBody() и передайте homeUiState.itemList в параметр itemList.

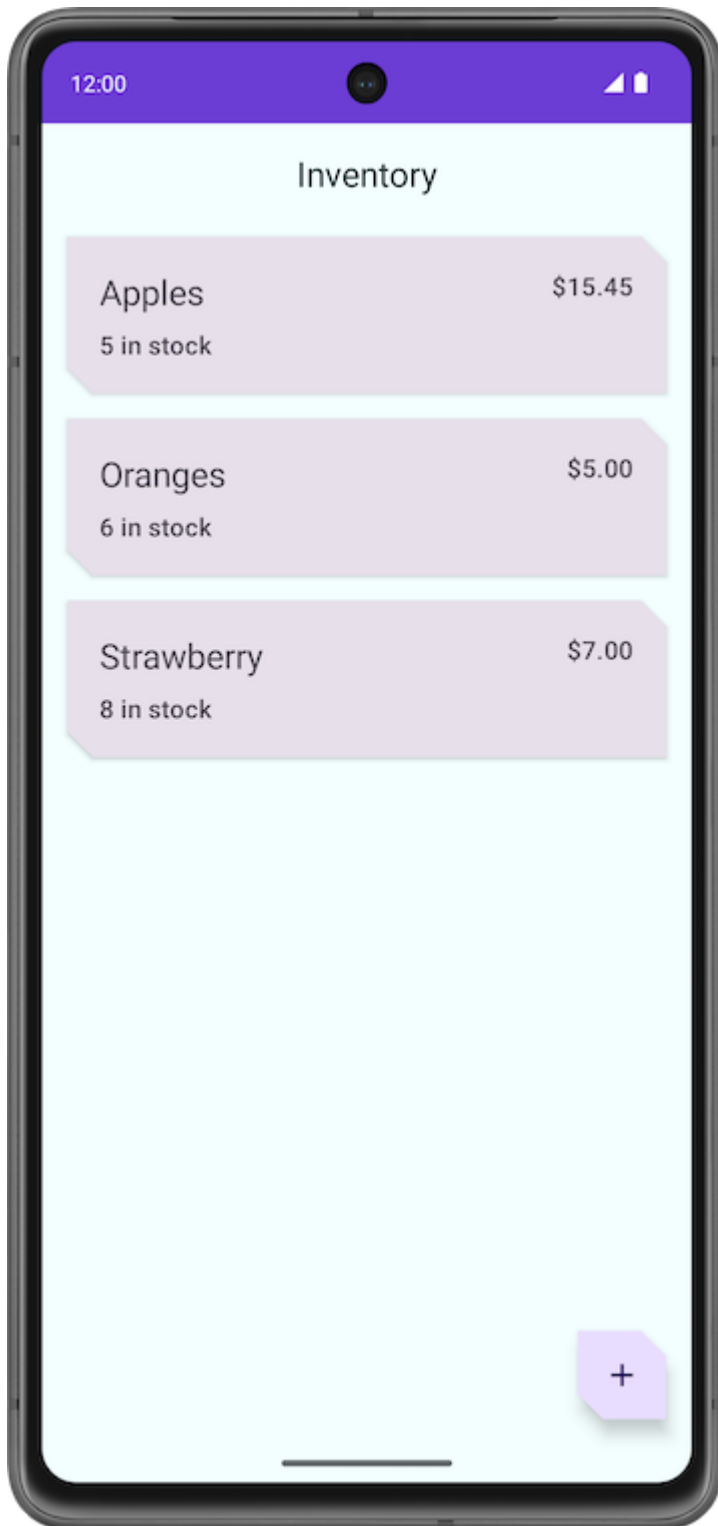
```

``kt
HomeBody(
    itemList = homeUiState.itemList,
    onItemClick = navigateToItemUpdate,

```

```
modifier = modifier.padding(innerPadding)  
)
```

Запустите приложение. Обратите внимание, что в списке инвентаря отображаются те предметы, которые вы сохранили в базе данных приложения. Если список пуст, добавьте несколько предметов инвентаря в базу данных приложения.



5. Протестируйте свою базу данных

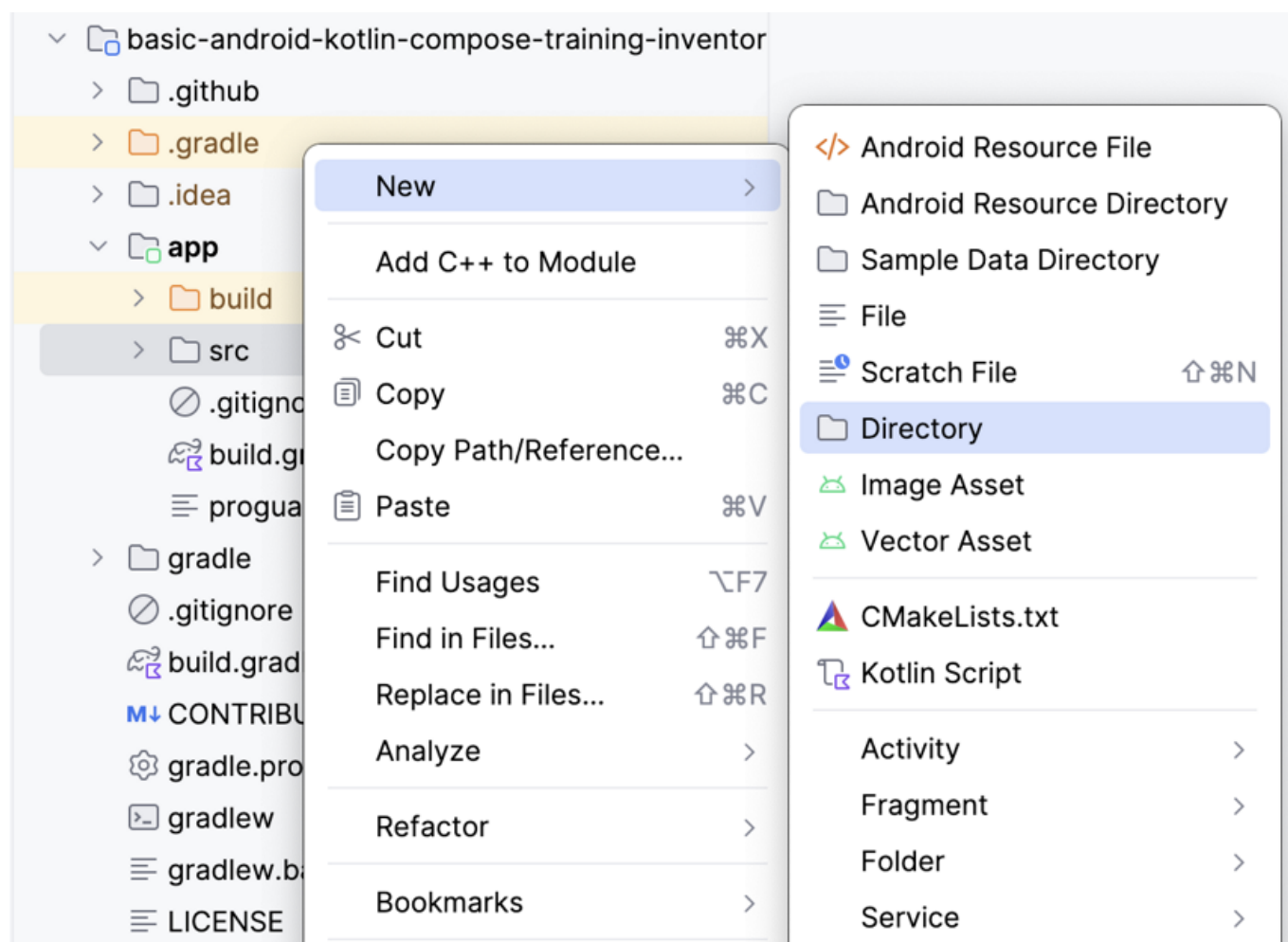
В предыдущих кодолабораториях обсуждалась важность тестирования кода. В этом задании вы добавите несколько модульных тестов для проверки ваших DAO-запросов, а затем будете добавлять новые тесты по мере продвижения по кодолабу.

Рекомендуемый подход к тестированию реализации базы данных - написание JUnit-тестов, которые выполняются на устройстве Android. Поскольку эти тесты не требуют создания активности, они выполняются быстрее, чем тесты пользовательского интерфейса.

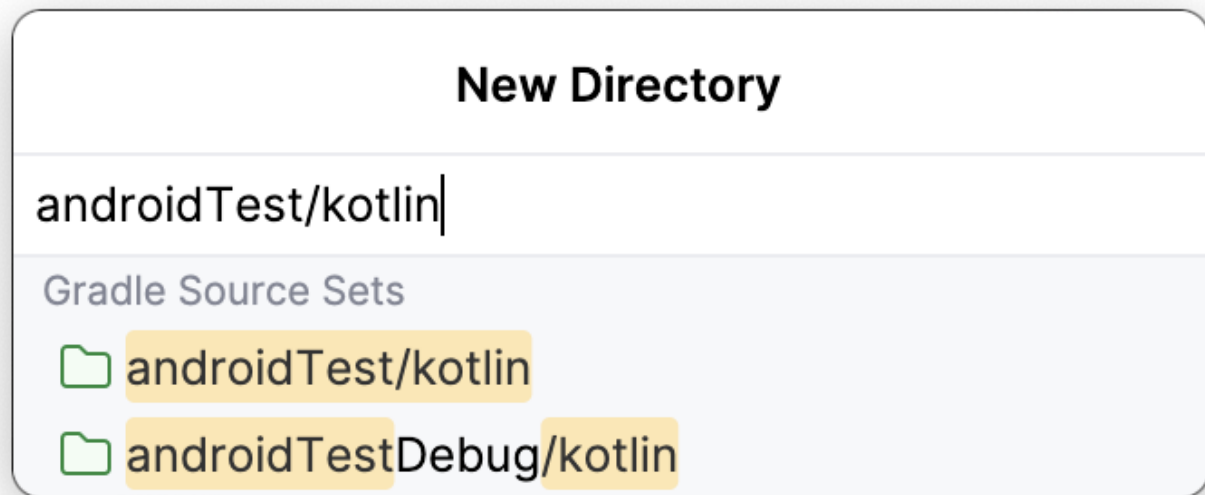
В файле `build.gradle.kts` (Модуль `:app`) обратите внимание на следующие зависимости для Espresso и JUnit.

```
// Тестирование
androidTestImplementation(«androidx.test.espresso:espresso-core:3.5.1»)
androidTestImplementation(«androidx.test.ext:junit:1.1.5»)
```

Переключитесь в режим просмотра проекта и щелкните правой кнопкой мыши на `src > New > Directory`, чтобы создать набор тестовых исходников для ваших тестов.



Выберите `androidTest/kotlin` во всплывающем окне `New Directory`.



Создайте класс Kotlin под названием ItemDaoTest.kt. Аннотируйте класс ItemDaoTest с помощью `@RunWith(AndroidJUnit4::class)`. Теперь ваш класс выглядит примерно так, как показано в следующем примере:

```
``kt package com.example.inventory
```

```
import androidx.test.ext.junit.runners.AndroidJUnit4 import org.junit.runner.RunWith
```

```
@RunWith(AndroidJUnit4::class) class ItemDaoTest { }
```

Внутри класса добавьте приватные переменные типа ItemDao и InventoryDatabase.

```
``kt
import com.example.inventory.data.InventoryDatabase
import com.example.inventory.data.ItemDao

private lateinit var itemDao: ItemDao
private lateinit var inventoryDatabase: InventoryDatabase
```

Добавьте функцию для создания базы данных и аннотируйте ее `@Before`, чтобы она запускалась перед каждым тестом. Внутри метода инициализируйте itemDao.

```
``kt import android.content.Context import androidx.room.Room import
import androidx.test.core.app.ApplicationProvider import org.junit.Before
```

```
@Before fun createDb() { val context: Context = ApplicationProvider.getApplicationContext() // Используем
базу данных в памяти, потому что информация, хранящаяся здесь, исчезает, когда // когда процесс убит.
inventoryDatabase = Room.inMemoryDatabaseBuilder(context, InventoryDatabase::class.java) // Разрешаем
запросы к главному потоку, просто для тестирования. .allowMainThreadQueries() .build() itemDao =
inventoryDatabase.itemDao() }
```

В этой функции вы используете базу данных in-memory и не сохраняете ее на диске. Для этого вы используете функцию `inMemoryDatabaseBuilder()`. Вы делаете это потому, что информация не должна сохраняться, а скорее должна быть удалена при завершении процесса. Вы выполняете DAO-запросы в главном потоке с помощью функции `.allowMainThreadQueries()`, просто для тестирования.

Добавьте еще одну функцию для закрытия базы данных. Аннотируйте ее с помощью `@After`, чтобы она закрывала базу данных и запускалась после каждого теста.

```
```kt
import org.junit.After
import java.io.IOException

@After
@Throws(IOException::class)
fun closeDb() {
 inventoryDatabase.close()
}
```

Объявите элементы в классе `ItemDaoTest` для базы данных, как показано в следующем примере кода:

```
```kt import com.example.inventory.data.Item
```

```
private var item1 = Item(1, «Apples», 10.0, 20) private var item2 = Item(2, «Бананы», 15.0, 97)
```

Добавьте служебные функции для добавления в базу данных одного, а затем двух элементов. Позже вы используете эти функции в своем тесте. Пометьте их как приостановленные, чтобы они могли выполняться в корутине.

```
```kt
private suspend fun addOneItemToDb() {
 itemDao.insert(item1)
}

private suspend fun addTwoItemsToDb() {
 itemDao.insert(item1)
 itemDao.insert(item2)
}
```

Напишите тест для вставки одного элемента в базу данных, `insert()`. Назовите тест `daoInsert_insertsItemIntoDB` и аннотируйте его с помощью `@Test`.

```
```kt import kotlinx.coroutines.flow.first import kotlinx.coroutines.runBlocking import
org.junit.Assert.assertEquals import org.junit.Test
```

```
@Test @Throws(Exception::class) fun daoInsert_insertsItemIntoDB() = runBlocking { addOneItemToDb() val allItems = itemDao.getAllItems().first() assertEquals(allItems[0], item1) }
```

В этом тесте вы используете служебную функцию `addOneItemToDb()` для добавления одного элемента в базу данных. Затем вы читаете первый элемент в базе данных. С помощью `assertEquals()` вы сравниваете ожидаемое значение с фактическим. Вы запускаете тест в новой корутине с `runBlocking{}`. Эта настройка - причина, по которой вы помечаете служебные функции как приостановленные.

Запустите тест и убедитесь, что он прошел.

Напишите еще один тест для `getAllItems()` из базы данных. Назовите тест `daoGetAllItems_returnsAllItemsFromDB`.

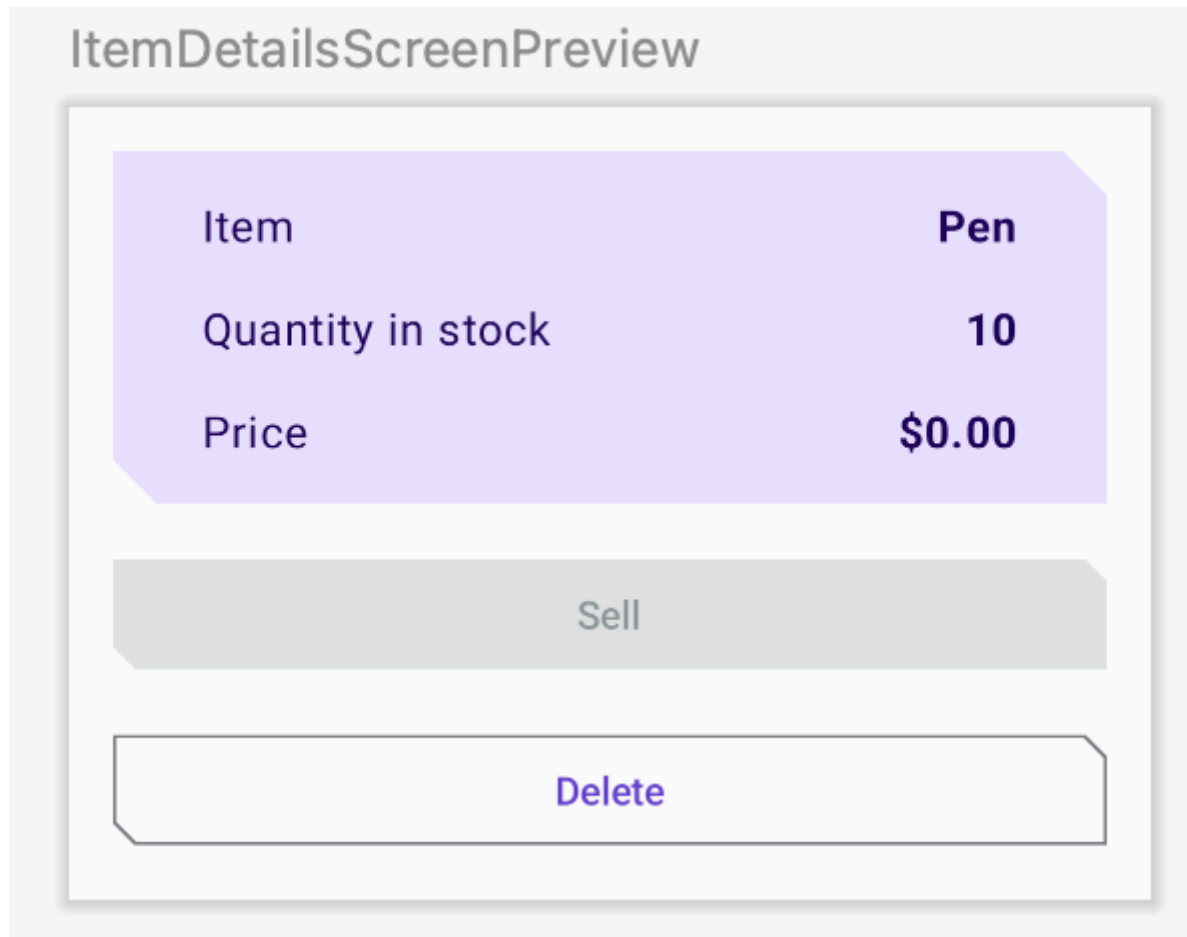
```
``kt
@Test
@Throws(Exception::class)
fun daoGetAllItems_returnsAllItemsFromDB() = runBlocking {
    addTwoItemsToDb()
    val allItems = itemDao.getAllItems().first()
    assertEquals(allItems[0], item1)
    assertEquals(allItems[1], item2)
}
```

В приведенном выше тесте вы добавляете два элемента в базу данных внутри корутины. Затем вы считываете эти два элемента и сравниваете их с ожидаемыми значениями.

6. Отображение деталей элемента

В этой задаче вы считываете и отображаете информацию о сущности на экране `Item Details`. Вы используете состояние пользовательского интерфейса элемента, такое как название, цена и количество, из базы данных приложения инвентаризации и отображаете их на экране `Item Details` с помощью составного элемента `ItemDetailsScreen`. Композитная функция `ItemDetailsScreen` уже написана для вас и содержит три композитных элемента `Text`, которые отображают сведения об элементе.

`ui/item/ItemDetailsScreen.kt` Этот экран является частью начального кода и отображает детали предметов, которые вы увидите в последующем уроке. В этом кодовом уроке вы не будете работать с этим экраном. `ItemDetailsViewModel.kt` - это соответствующая `ViewModel` для этого экрана.



В составной функции `HomeScreen` обратите внимание на вызов функции `HomeBody()`. `navigateToItemUpdate` передается параметру `onItemClick`, который вызывается при щелчке на любом элементе списка.

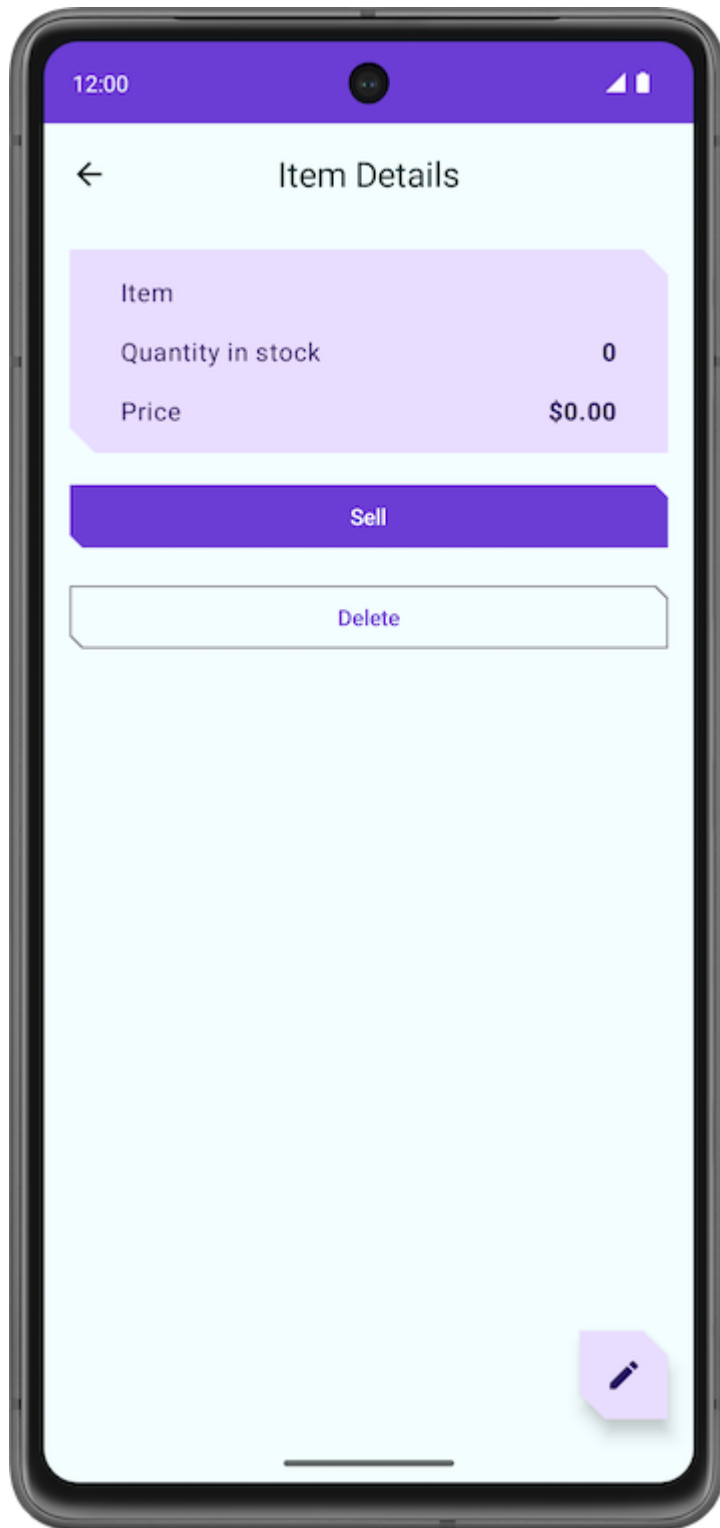
```
``kt // Нет необходимости копировать HomeBody( itemList = homeUiState.itemList, onItemClick =
navigateToItemUpdate, модификатор = модификатор .padding(innerPadding) .fillMaxSize() )
```

Откройте `ui/navigation/InventoryNavGraph.kt` и обратите внимание на параметр `navigateToItemUpdate` в составной части `HomeScreen`. Этот параметр определяет место назначения для навигации как экран подробной информации об элементе.

```
``kt
// Нет необходимости копировать
HomeScreen(
    navigateToItemEntry = { navController.navigate(ItemEntryDestination.route) },
    navigateToItemUpdate = {
        navController.navigate(«${ItemDetailsDestination.route}/${it}»)
    }
)
```

Эта часть функциональности `onItemClick` уже реализована для вас. Когда вы нажимаете на элемент списка, приложение переходит на экран подробной информации об элементе.

Щелкните любой элемент в списке инвентаря, чтобы увидеть экран сведений об элементе с пустыми полями.



Чтобы заполнить текстовые поля деталями элемента, необходимо собрать состояние пользовательского интерфейса в функции `ItemDetailsScreen()`.

В `UI/Item/ItemDetailsScreen.kt` добавьте новый параметр к `ItemDetailsScreen` composable типа `ItemDetailsViewModel` и используйте фабричный метод для его инициализации.

```
``kt import androidx.lifecycle.viewmodel.compose.viewModel import  
com.example.inventory.ui.AppViewModelProvider
```

```
@Composable fun ItemDetailsScreen( navigateToEditItem: (Int) -> Unit, navigateBack: () -> Unit,  
модификатор: Modifier = Modifier, viewModel: ItemDetailsViewModel = viewModel(factory =  
AppViewModelProvider.Factory) )
```

Внутри композита `ItemDetailsScreen()` создайте `val` под названием `uiState` для сбора состояния пользовательского интерфейса. Используйте функцию `collectAsState()` для сбора потока состояний `uiState` и представления его последнего значения через `State`. Android Studio выдает ошибку неразрешенной ссылки.

```
``kt
import androidx.compose.runtime.collectAsState

val uiState = viewModel.uiState.collectAsState()
```

Чтобы устранить ошибку, создайте `val` с именем `uiState` типа `StateFlow` в классе `ItemDetailsViewModel`. Получите данные из хранилища элементов и сопоставьте их с `ItemDetailsUiState` с помощью функции расширения `toItemDetails()`. Функция расширения `Item.toItemDetails()` уже написана для вас в составе стартового кода.

```
``kt import androidx.lifecycle.viewModelScope import kotlinx.coroutines.flow.SharingStarted import
kotlinx.coroutines.flow.StateFlow импорт kotlinx.coroutines.flow.filterNotNull импорт
kotlinx.coroutines.flow.map import kotlinx.coroutines.flow.stateIn
```

```
val uiState: StateFlow = itemsRepository.getItemStream(itemId) .filterNotNull() .map {
ItemDetailsUiState(itemDetails = it.toItemDetails()) }.stateIn( scope = viewModelScope, started =
SharingStarted.WhileSubscribed(TIMEOUT_MILLIS), initialValue = ItemDetailsUiState() )
```

Передаем `ItemsRepository` в `ItemDetailsViewModel`, чтобы разрешить ошибку `Unresolved reference: itemsRepository`.

```
``kt
class ItemDetailsViewModel(
    savedStateHandle: SavedStateHandle,
    private val itemsRepository: ItemsRepository
) : ViewModel() {
```

В `ui/AppViewModelProvider.kt` обновите инициализатор для `ItemDetailsViewModel`, как показано в следующем фрагменте кода:

```
``kt инициализатор { ItemDetailsViewModel( this.createSavedStateHandle(),
inventoryApplication().container.itemsRepository ) }
```

Вернитесь к файлу `ItemDetailsScreen.kt` и обратите внимание на то, что ошибка в составном элементе `ItemDetailsScreen()` устранена. В составной части `ItemDetailsScreen()` обновите вызов функции `ItemDetailsBody()` и передайте в аргумент `itemUiState.value` значение `uiState`.

```
``kt
ItemDetailsBody(
    itemUiState = uiState.value,
    onSellItem = { },
    onDelete = { },
    modifier = modifier.padding(innerPadding)
)
```

Наблюдайте за реализацией `ItemDetailsBody()` и `ItemInputForm()`. Вы передаете текущий выбранный элемент из `ItemDetailsBody()` в `ItemDetails()`.

``kt // Нет необходимости копировать поверх

```
@Composable private fun ItemDetailsBody( itemUiState: ItemUiState, onSellItem: () -> Unit, onDelete: () ->
Unit, модификатор: Модификатор = Модификатор ) { Column( //... ) { var deleteConfirmationRequired by
rememberSaveable { mutableStateOf(false) } ItemDetails( item = itemDetailsUiState.itemDetails.toItem(),
modifier = Modifier.fillMaxWidth() )
```

```
//...
}
```

Запустите приложение. При щелчке на любом элементе списка на экране Inventory отображается экран Item Details.

Обратите внимание, что экран больше не пустой. На нем отображаются сведения об объекте, полученные из базы данных инвентаризации.

Нажмите кнопку «Продать». Ничего не происходит!

В следующем разделе вы реализуете функциональность кнопки «Продать».

7. Реализация экрана подробной информации об элементе

ui/item/ItemEditScreen.kt

Экран редактирования элемента уже предоставлен вам в качестве части стартового кода.

Этот макет содержит текстовые поля для редактирования деталей любого нового предмета инвентаря.

Код этого приложения все еще не полностью функционален. Например, на экране «Детали товара» при нажатии кнопки «Продать» количество товара на складе не уменьшается. Когда вы нажимаете кнопку Delete, приложение выдает диалог подтверждения. Однако, когда вы выбираете кнопку «Да», приложение не удаляет

товар.

Наконец, кнопка FAB aad0ce469e4a3a12.png открывает пустой экран редактирования элемента.

В этом разделе вы реализуете функциональность кнопок Sell, Delete и FAB.

8. Реализация продажи товара

В этом разделе вы расширяете возможности приложения для реализации функции продажи. Это обновление включает в себя следующие задачи:

Добавьте тест для функции DAO для обновления сущности.

Добавьте функцию в ItemDetailsViewModel для уменьшения количества и обновления сущности в базе данных приложения.

Отключите кнопку «Продать», если количество товара равно нулю.

В ItemDaoTest.kt добавьте функцию daoUpdateItems_updatesItemsInDB() без параметров. Аннотируйте ее с помощью @Test и @Throws(Exception::class).

```
``kt
@Test
@Throws(Exception::class)
fun daoUpdateItems_updatesItemsInDB()
```

Определите функцию и создайте блок runBlocking. Внутри него вызовите addTwoItemsToDb().

```
``kt fun daoUpdateItems_updatesItemsInDB() = runBlocking { addTwoItemsToDb() }
```

Обновляем две сущности с разными значениями, вызывая itemDao.update.

```
``kt
itemDao.update(Item(1, «Яблоки», 15.0, 25))
itemDao.update(Item(2, «Бананы», 5.0, 50))
```

Получите сущности с помощью itemDao.getAllItems(). Сравните их с обновленной сущностью и утвердите.

```
``kt val allItems = itemDao.getAllItems().first() assertEquals(allItems[0], Item(1, «Apples», 15.0, 25))
assertEquals(allItems[1], Item(2, «Bananas», 5.0, 50))
```

Убедитесь, что готовая функция выглядит следующим образом:


```

```kt
@Test
@Throws(Exception::class)
fun daoUpdateItems_updatesItemsInDB() = runBlocking {
 addTwoItemsToDb()
 itemDao.update(Item(1, «Яблоки», 15.0, 25))
 itemDao.update(Item(2, «Бананы», 5.0, 50))

 val allItems = itemDao.getAllItems().first()
 assertEquals(allItems[0], Item(1, «Яблоки», 15.0, 25))
 assertEquals(allItems[1], Item(2, «Bananas», 5.0, 50))
}

```

Запустите тест и убедитесь, что он прошел. Добавьте функцию в ViewModel В ItemDetailsViewModel.kt, внутри класса ItemDetailsViewModel, добавьте функцию reduceQuantityByOne() без параметров.

```

fun reduceQuantityByOne() {
}

```

Внутри функции запустите корутину с помощью viewModelScope.launch{}

**Примечание:** Вы должны запускать операции с базой данных внутри корутины.

```

```kt import kotlinx.coroutines.launch import androidx.lifecycle.viewModelScope

```

```

viewModelScope.launch {}

```

Внутри блока запуска создайте val под названием currentItem и установите его в uiState.value.toItem().

```

```kt
val currentItem = uiState.value.toItem()

```

Значение uiState.value имеет тип ItemUiState. Вы преобразуете его к типу сущности Item с помощью функции расширения toItem().

Добавьте оператор if для проверки того, что качество больше 0. Вызовите updateItem() для itemsRepository и передайте обновленный currentItem. Используйте copy() для обновления значения количества, чтобы функция выглядела следующим образом:

```

```kt fun reduceQuantityByOne() { viewModelScope.launch { val currentItem = uiState.value.itemDetails.toItem()
if (currentItem.quantity > 0) { itemsRepository.updateItem(currentItem.copy(quantity = currentItem.quantity - 1)) } } }

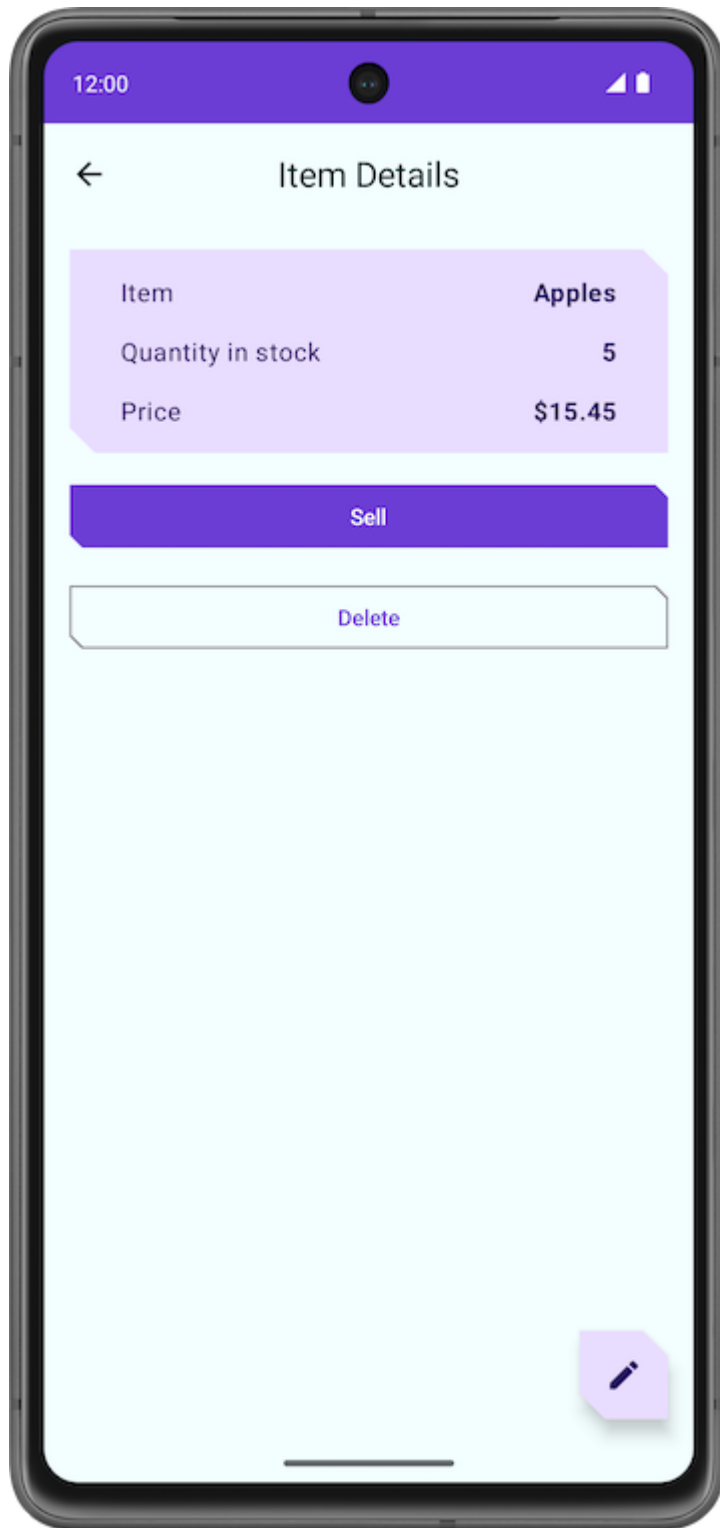
```

Вернитесь к файлу ItemDetailsScreen.kt.

В составной части `ItemDetailsScreen` перейдите к вызову функции `ItemDetailsBody()`. В лямбде `onSellItem` вызовите `viewModel.reduceQuantityByOne()`.

```
``кТ
ItemDetailsBody(
    itemUiState = uiState.value,
    onSellItem = { viewModel.reduceQuantityByOne() },
    onDelete = { },
    modifier = modifier.padding(innerPadding)
)
```

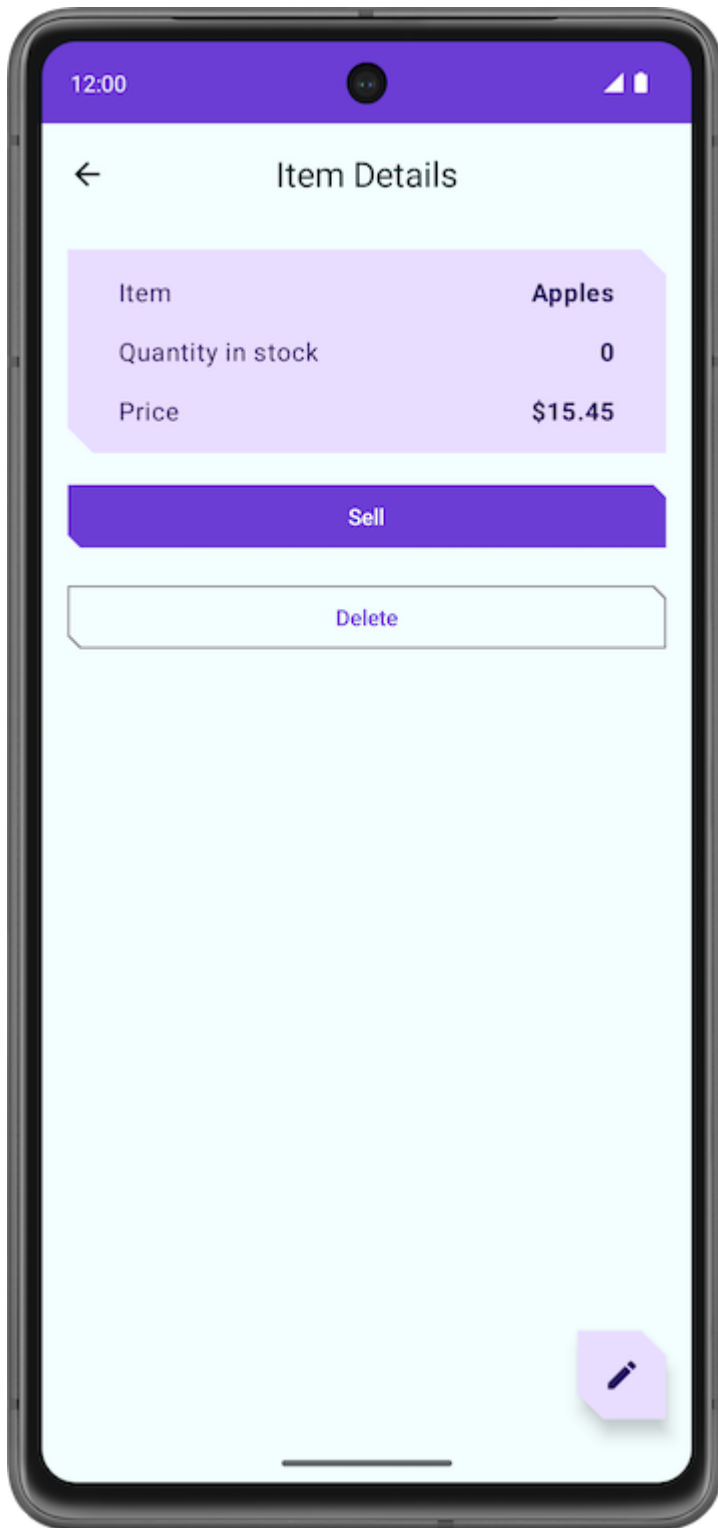
Запустите приложение. На экране «Инвентарь» щелкните элемент списка. Когда появится экран «Сведения об элементе», нажмите «Продать» и обратите внимание, что значение количества уменьшилось на единицу.



На экране «Сведения об элементе» постоянно нажимайте кнопку «Продать», пока количество не станет равным нулю.

Совет: Чтобы сэкономить время, вы можете использовать для этой задачи предмет с низким количеством. Если ни один из ваших элементов не имеет низкого количества, вы можете создать новый элемент с низким количеством.

После того как количество достигнет нуля, снова нажмите Продать. Визуальных изменений не произойдет, потому что функция `reduceQuantityByOne()` проверяет, больше ли количество, чем ноль, перед обновлением количества.



Чтобы дать пользователям лучшую обратную связь, вы можете отключить кнопку «Продать», когда нет товара для продажи.

В классе `ItemDetailsViewModel` установите значение `outOfStock` на основе `it.quantity` в преобразовании `map`.

```
kt val uiState: StateFlow = itemsRepository.getItemStream(itemId) .filterNotNull() .map {  
ItemDetailsUiState(outOfStock = it.quantity <= 0, itemDetails = it.toItemDetails()) }.stateIn( //... )
```

Запустите ваше приложение. Обратите внимание, что приложение отключает кнопку

«Продать», когда количество товара на складе равно нулю.

Поздравляем вас с реализацией функции продажи товаров в вашем приложении!

Удаление сущности товара

Как и в предыдущей задаче, вы должны расширить возможности своего приложения, реализовав функцию удаления товара. Эту функцию реализовать гораздо проще, чем функцию продажи. Процесс включает в себя следующие задачи:

Добавьте тест для DAO-запроса на удаление.

Добавьте функцию в класс `ItemDetailsViewModel` для удаления сущности из базы данных.

Обновить составной элемент `ItemDetailsBody`.

Добавьте тест DAO

В `ItemDaoTest.kt` добавьте тест `daoDeleteItems_deletesAllItemsFromDB()`.

```
``kt
@Test
@Throws(Exception::class)
fun daoDeleteItems_deletesAllItemsFromDB()
```

Запускаем корутину с `runBlocking {}`.

```
fun daoDeleteItems_deletesAllItemsFromDB() = runBlocking {
}
```

Добавьте два элемента в базу данных и вызовите `itemDao.delete()` для этих двух элементов, чтобы удалить их из базы данных.

```
addTwoItemsToDb()
itemDao.delete(item1)
itemDao.delete(item2)
```

Получите сущности из базы данных и проверьте, что список пуст. Выполненный тест должен выглядеть следующим образом:

```
``kt import org.junit.Assert.assertTrue
```

```
@Test @Throws(Exception::class) fun daoDeleteItems_deletesAllItemsFromDB() = runBlocking {
addTwoItemsToDb() itemDao.delete(item1) itemDao.delete(item2) val allItems = itemDao.getAllItems().first()
assertTrue(allItems.isEmpty()) }
```

Добавьте функцию удаления в `ItemDetailsViewModel`

В `ItemDetailsViewModel` добавьте новую функцию `deleteItem()`, которая не принимает никаких параметров и ничего не возвращает.

Внутри функции `deleteItem()` добавьте вызов функции `itemsRepository.deleteItem()` и передайте в нее `uiState.value.toItem()`.

```
```kt
suspend fun deleteItem() {
 itemsRepository.deleteItem(uiState.value.itemDetails.toItem())
}
```

В этой функции вы конвертируете `uiState` из типа `itemDetails` в тип сущности `Item` с помощью функции расширения `toItem()`.

В композиции `ui/item/ItemDetailsScreen` добавьте `val` под названием `coroutineScope` и установите его в `rememberCoroutineScope()`. Этот подход возвращает область видимости корутины, привязанную к композиции, в которой она была вызвана (композиция `ItemDetailsScreen`).

```
```kt import androidx.compose.runtime.rememberCoroutineScope
```

```
val coroutineScope = rememberCoroutineScope()
```

Прокрутите страницу до функции `ItemDetailsBody()`.

Запустите корутину с `coroutineScope` внутри лямбды `onDelete`.

Внутри блока запуска вызовите метод `deleteItem()` на `viewModel`.

```
```kt
import kotlinx.coroutines.launch

ItemDetailsBody(
 itemUiState = uiState.value,
 onSellItem = { viewModel.reduceQuantityByOne() },
 onDelete = {
 coroutineScope.launch {
 viewModel.deleteItem()
 }
 },
 modifier = modifier.padding(innerPadding)
)
```

После удаления предмета перейдите обратно на экран инвентаря. Вызовите `navigateBack()` после вызова функции `deleteItem()`.

```
```kt onDelete = { coroutineScope.launch { viewModel.deleteItem() navigateBack() }
```

Все еще находясь в файле `ItemDetailsScreen.kt`, прокрутите страницу до функции `ItemDetailsBody()`.

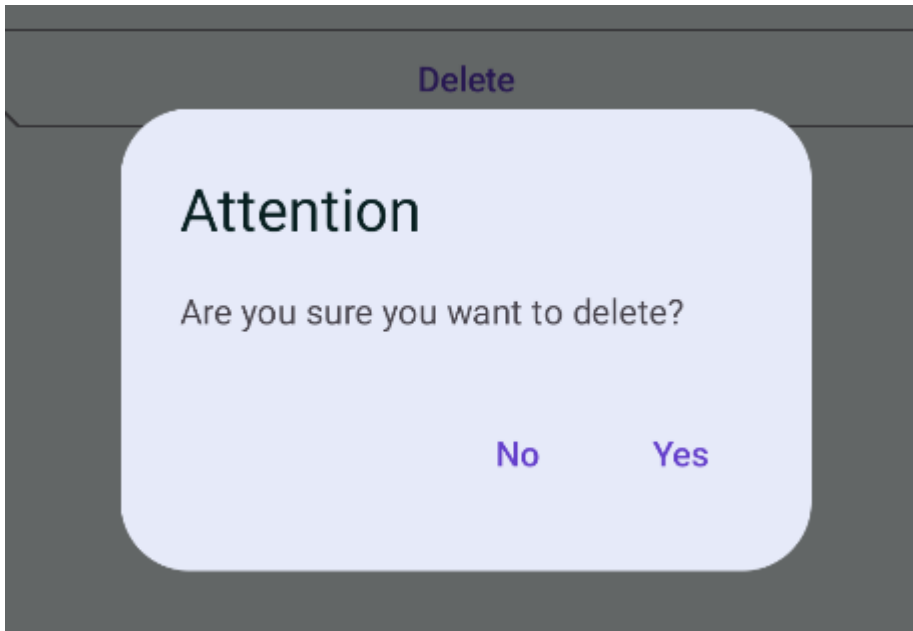
Эта функция является частью начального кода. Она отображает диалог предупреждения, чтобы получить подтверждение пользователя перед удалением элемента, и вызывает функцию `deleteItem()`, когда вы нажимаете Yes.

```
``kt
// Не нужно копировать

@Composable
private fun ItemDetailsBody(
    itemUiState: ItemUiState,
    onSellItem: () -> Unit,
    onDelete: () -> Unit,
    модификатор: Модификатор = Модификатор
) {
    Column(
        /*...*/
    ) {
        //...

        if (deleteConfirmationRequired) {
            DeleteConfirmationDialog(
                onDeleteConfirm = {
                    deleteConfirmationRequired = false
                    onDelete()
                },
                //...
            )
        }
    }
}
```

When you tap No, the app closes the alert dialog. The `showConfirmationDialog()` function displays the following alert:



Запустите приложение. Выберите элемент списка на экране «Инвентарь». На экране «Сведения об элементе» нажмите Удалить. Нажмите Да в диалоговом окне оповещения, и приложение вернется на экран «Инвентаризация». Убедитесь, что удаленный элемент больше не находится в базе данных приложения. Поздравляем с внедрением функции удаления!



Редактирование сущности элемента Как и в предыдущих разделах, в этом разделе вы добавите в приложение еще одну функцию, позволяющую редактировать сущность элемента.

Вот краткое описание шагов по редактированию сущности в базе данных приложения:

Добавьте тест к DAO-запросу «Получить элемент». Заполните текстовые поля и экран Edit Item деталями сущности. Обновите сущность в базе данных с помощью Room. Добавьте тест DAO В файл ItemDaoTest.kt добавьте тест `daoGetItem_returnsItemFromDB()`.

```
@Test
@Throws(Exception::class)
fun daoGetItem_returnsItemFromDB()
```

Определите функцию. Внутри корутины добавьте один элемент в базу данных.

```
`kt @Test @Throws(Exception::class) fun daoGetItem_returnsItemFromDB() = runBlocking {
addOneItemToDb() }
```

Получаем сущность из базы данных с помощью функции `itemDao.getItem()` и устанавливаем ее в `val` с именем `item`.

```
`kt
val item = itemDao.getItem(1)
```


Сравните фактическое значение с полученным и выполните проверку с помощью функции `assertEquals()`. Ваш завершённый тест выглядит следующим образом:

```
``kt @Test @Throws(Exception::class) fun daoGetItem_returnsItemFromDB() = runBlocking {
    addOneItemToDb() val item = itemDao.getItem(1) assertEquals(item.first(), item1) }
```

Запустите тест и убедитесь, что он прошёл.

Заполнение текстовых полей

Если вы запустите приложение, перейдете к экрану `Item Details`, а затем щелкните FAB, вы увидите, что теперь заголовок экрана - `Edit Item`. Однако все текстовые поля пусты. В этом шаге вы заполните текстовые поля на экране «Редактирование элемента» сведениями о сущности.

```
<div style=«display:flex»>
    <img src=«https://developer.android.com/static/codelabs/basic-android-kotlin-compose-update-data-room/img/3aac7e2c9e7a04b6_856.png»/>
    <img src=«https://developer.android.com/static/codelabs/basic-android-kotlin-compose-update-data-room/img/cdccb3a8931b4a3_856.png»/>
</div>
```

В файле `ItemDetailsScreen.kt` прокрутите страницу до композита `ItemDetailsScreen`. В функции `FloatingActionButton()` измените аргумент `onClick`, чтобы он включал `uiState.value.itemDetails.id`, который является идентификатором выбранной сущности. Вы используете этот идентификатор для получения сведений о сущности.

```
``kt
FloatingActionButton(
    onClick = { navigateToEditItem(uiState.value.itemDetails.id) },
    modifier = /*...*/
)
```

In the `ItemEditViewModel` class, add an `init` block.

```
init {

}
```

Внутри блока `init` запустите корутину с помощью `viewModelScope.launch`.

```
import kotlinx.coroutines.launch

viewModelScope.launch { }
```

Внутри блока запуска получите данные о сущности с помощью `itemsRepository.getItemStream(itemId)`.

```
``kt import androidx.lifecycle.viewModelScope import kotlinx.coroutines.flow.filterNotNull import
kotlinx.coroutines.flow.first
```

```
init { viewModelScope.launch { itemUiState = itemsRepository.getItemStream(itemId) .filterNotNull() .first()
.toItemUiState(true) } }
```

В этом блоке запуска вы добавляете фильтр для возврата потока, который содержит только значения, не являющиеся null. С помощью функции toItemUiState() вы преобразуете сущность item в ItemUiState. Вы передаете значение actionEnabled как true, чтобы включить кнопку Save.

Чтобы устранить ошибку Unresolved reference: itemsRepository, необходимо передать ItemsRepository в качестве зависимости в модель представления.

Добавьте параметр конструктора в класс ItemEditViewModel.

```
``kt
class ItemEditViewModel(
    savedStateHandle: SavedStateHandle,
    private val itemsRepository: ItemsRepository
)
```

В файле AppViewModelProvider.kt, в инициализаторе ItemEditViewModel, добавьте объект ItemsRepository в качестве аргумента.

```
initializer {
    ItemEditViewModel(
        this.createSavedStateHandle(),
        inventoryApplication().container.itemsRepository
    )
}
```

Запустите приложение. Перейдите в раздел «Сведения об изделии» и нажмите FAB. Обратите внимание, что в полях появляются сведения о товаре. Отредактируйте количество товара на складе или любое другое поле и нажмите Сохранить. Ничего не происходит! Это происходит потому, что вы не обновляете сущность в базе данных приложения. Вы исправите это в следующем разделе.



Обновление сущности с помощью Room В этой заключительной задаче вы добавляете последние части кода для реализации функциональности обновления. Вы определяете необходимые функции в ViewModel и используете их в ItemEditScreen.

Снова настало время кодирования!

В класс ItemEditViewModel добавьте функцию updateUiState(), которая принимает объект ItemUiState и ничего не возвращает. Эта функция обновляет itemUiState новыми значениями, которые вводит

пользователь.

```
fun updateUiState(itemDetails: ItemDetails) {
    itemUiState =
        ItemUiState(itemDetails = itemDetails, isEntryValid =
            validateInput(itemDetails))
}
```

В этой функции вы присваиваете переданный элемент `itemDetails` состоянию `itemUiState` и обновляете значение `isEntryValid`. Приложение включает кнопку `Save`, если значение `itemDetails` равно `true`. Вы устанавливаете это значение в `true` только в том случае, если вводимые пользователем данные действительны.

Перейдите к файлу `ItemEditScreen.kt`. В составной части `ItemEditScreen` прокрутите вниз до вызова функции `ItemEntryBody()`. Установите значение аргумента `onItemValueChange` в новую функцию `updateUiState`.

```
``kt ItemEntryBody( itemUiState = viewModel.itemUiState, onItemValueChange = viewModel::updateUiState,
onSaveClick = { }, modifier = modifier.padding(innerPadding) )
```

Запустите приложение.

Перейдите на экран редактирования элемента.

Сделайте одно из значений сущности пустым, чтобы оно стало недействительным.

Обратите внимание, что кнопка `Save` отключается автоматически.

```
<div style=«display:flex»>
    <img src=«https://developer.android.com/static/codelabs/basic-android-kotlin-
compose-update-data-room/img/d368151eb7b198cd_856.png»/>
    <img src=«https://developer.android.com/static/codelabs/basic-android-kotlin-
compose-update-data-room/img/427ff7e2bf45f6ca_856.png»/>
    <img src=«https://developer.android.com/static/codelabs/basic-android-kotlin-
compose-update-data-room/img/9aa8fa86a928e1a6_856.png»/>
</div>
```

Вернитесь в класс `ItemEditViewModel` и добавьте функцию приостановки `updateItem()`, которая ничего не принимает. Вы используете эту функцию для сохранения обновленной сущности в базе данных `Room`.

```
``kt
suspend fun updateItem() {
}
```

Внутри функции `getUpdatedItemEntry()` добавьте условие `if` для проверки вводимых пользователем данных с помощью функции `validateInput()`. Выполните вызов функции `updateItem()` в хранилище

itemsRepository, передав в нее itemUiState.itemDetails.toItem(). Объекты, которые могут быть добавлены в базу данных Room, должны иметь тип Item. Завершенная функция выглядит следующим образом:

```
``kt suspend fun updateItem() { if (validateInput(itemUiState.itemDetails)) {
itemsRepository.updateItem(itemUiState.itemDetails.toItem()) } }
```

Вернитесь к составному экрану ItemEditScreen. Для вызова функции updateItem() вам понадобится область видимости coroutine. Создайте val под названием coroutineScope и установите для него значение rememberCoroutineScope().

```
``kt
import androidx.compose.runtime.rememberCoroutineScope

val coroutineScope = rememberCoroutineScope()
```

В вызове функции ItemEntryBody() обновите аргумент функции onSaveClick, чтобы запустить корутину в coroutineScope. Внутри блока запуска вызовите updateItem() для viewModel и перейдите обратно.

```
``kt import kotlinx.coroutines.launch
```

```
onSaveClick = { coroutineScope.launch { viewModel.updateItem() navigateBack() } },
```

Завершенный вызов функции ItemEntryBody() выглядит следующим образом:

```
``kt
ItemEntryBody(
    itemUiState = viewModel.itemUiState,
    onItemValueChange = viewModel::updateUiState,
    onSaveClick = {
        coroutineScope.launch {
            viewModel.updateItem()
            navigateBack()
        }
    },
    modifier = modifier.padding(innerPadding)
)
```

Запустите приложение и попробуйте отредактировать элементы инвентаря. Теперь вы можете редактировать любой элемент в базе данных приложения Inventory.



Поздравляем вас с созданием первого приложения, использующего Room для управления базой данных!

Код решения

Код решения для этого урока находится в репозитории GitHub и в ветке, показанной ниже:

URL-адрес кода решения:

<https://github.com/google-developer-training/basic-android-kotlin-compose-training-inventory-app>